Conversation

- conversationId: 1bd226c2-a60b-4487-91ca-0ca6c7f15cec
- endpoint: Mistral
- title: Protein-Drug Affinity Prediction
- exportAt: 15:45:40 GMT+0200 (Eastern European Standard Time)

Options

- presetId: null
- model: devstral-latest
- resendFiles: true
- iconURL: Mistral
- spec: mistral-devstral-latest
- endpoint: Mistral
- endpointType: custom
- title: Protein-Drug Affinity Prediction

History

User
Hello, I'm working on a machine learning project which invloves a specfici set of proteins. My data set looks like following,

- 400 pairs of proteins and drugs
- 77 unique target proteins and 59 unique drug molecules
- Features for each pair in 'input.data'. shape: 400 x features
- Affinities (targets) in 'output.data'. shape: 400
- Pair identifiers in 'pairs.data' (protein_id, drug_id)

I need to:

1. Load and preprocess this data
2. Train a K-nearest neighbors regressor with K=10
3. Perform leave-one-out cross-validation using C-index metric
4. Calculate C-index score

please write me the code for the above steps. use pandas to load data, numpy for numpy array, sklearn libraries for k-NN and other algos and finally matplotlib for plotting out the graphs for

vsiualization.

Please along with the code explain the parts and their working.

Mistral

Protein-Drug Affinity Prediction with K-Nearest Neighbors

Here's a complete solution for your machine learning project, with explanations for each step:

```
import numpy as np
import pandas as pd
from sklearn.neighbors import KNeighborsRegressor
from sklearn.model_selection import LeaveOneOut
from sklearn.metrics import pairwise_distances
import matplotlib.pyplot as plt
from scipy.stats import rankdata

# Custom function to calculate C-index
def c_index(y_true, y_pred):
    """
    Calculate the concordance index (C-index) for evaluating prediction
performance.

    Parameters:
    y_true (array-like): True target values
    y_pred (array-like): Predicted target values

    Returns:
    float: C-index score (between 0 and 1, higher is better)
    """
    # Convert to numpy arrays if they aren't already
    y_true = np.asarray(y_true)
    y_pred = np.asarray(y_pred)

    # Initialize counters
    concordant = 0
    permissible = 0

    # Compare all possible pairs
    for i in range(len(y_true)):
        for j in range(i+1, len(y_true)):
            # Check if the pair is permissible (different true values)
            if y_true[i] != y_true[j]:
                permissible += 1
```

```
                # Check if the prediction order matches the true order
                if (y_pred[i] > y_pred[j] and y_true[i] > y_true[j]) or \
                   (y_pred[i] < y_pred[j] and y_true[i] < y_true[j]):
                    concordant += 1
                # If predictions are equal, it's neither concordant nor
discordant

    # Handle case where all pairs have equal true values
    if permissible == 0:
        return 0.0

    return concordant / permissible

# 1. Load and preprocess the data
def load_and_preprocess_data(input_file, output_file, pairs_file):
    """
    Load and preprocess the dataset.

    Parameters:
    input_file (str): Path to input features file
    output_file (str): Path to output affinities file
    pairs_file (str): Path to pairs identifiers file

    Returns:
    tuple: (X, y, pairs_df) where X is features, y is targets, pairs_df
contains identifiers
    """
    # Load the data using pandas
    X = pd.read_csv(input_file, header=None).values  # Features (400 x
features)
    y = pd.read_csv(output_file, header=None).values.ravel()  # Targets (400,)
    pairs_df = pd.read_csv(pairs_file, header=None, names=['protein_id',
'drug_id'])  # Pair identifiers

    # Standardize the features (important for distance-based algorithms like
KNN)
    X_mean = X.mean(axis=0)
    X_std = X.std(axis=0)
    X = (X - X_mean) / X_std

    return X, y, pairs_df

# 2. Train KNN regressor with K=10 and perform LOOCV
def knn_loocv_with_cindex(X, y, k=10):
    """
    Perform leave-one-out cross-validation with KNN regressor and evaluate
```

```
using C-index.

    Parameters:
    X (array-like): Input features
    y (array-like): Target values
    k (int): Number of neighbors for KNN

    Returns:
    tuple: (predictions, c_index_score)
    """
    # Initialize KNN regressor
    knn = KNeighborsRegressor(n_neighbors=k)

    # Initialize Leave-One-Out cross-validator
    loo = LeaveOneOut()

    # Store predictions
    predictions = np.zeros_like(y)

    # Perform LOOCV
    for train_index, test_index in loo.split(X):
        X_train, X_test = X[train_index], X[test_index]
        y_train = y[train_index]

        # Train the model
        knn.fit(X_train, y_train)

        # Make prediction for the left-out sample
        predictions[test_index] = knn.predict(X_test)

    # Calculate C-index
    c_index_score = c_index(y, predictions)

    return predictions, c_index_score

# 3. Visualization functions
def plot_true_vs_predicted(y_true, y_pred):
    """
    Plot true vs predicted values.

    Parameters:
    y_true (array-like): True target values
    y_pred (array-like): Predicted target values
    """
    plt.figure(figsize=(8, 6))
    plt.scatter(y_true, y_pred, alpha=0.5)
```

```
    plt.plot([y_true.min(), y_true.max()], [y_true.min(), y_true.max()], 'k--
', lw=2)
    plt.xlabel('True Affinity')
    plt.ylabel('Predicted Affinity')
    plt.title('True vs Predicted Affinity Values')
    plt.grid(True)
    plt.show()

def plot_residuals(y_true, y_pred):
    """
    Plot residuals (errors) of predictions.

    Parameters:
    y_true (array-like): True target values
    y_pred (array-like): Predicted target values
    """
    residuals = y_true - y_pred
    plt.figure(figsize=(8, 6))
    plt.scatter(y_pred, residuals, alpha=0.5)
    plt.axhline(y=0, color='r', linestyle='-')
    plt.xlabel('Predicted Affinity')
    plt.ylabel('Residuals (True - Predicted)')
    plt.title('Residual Plot')
    plt.grid(True)
    plt.show()

# Main execution
if __name__ == "__main__":
    # File paths (replace with your actual file paths)
    input_file = 'input.data'
    output_file = 'output.data'
    pairs_file = 'pairs.data'

    # 1. Load and preprocess data
    X, y, pairs_df = load_and_preprocess_data(input_file, output_file,
pairs_file)
    print(f"Data loaded. Features shape: {X.shape}, Targets shape: {y.shape}")

    # 2. Train KNN with LOOCV and calculate C-index
    predictions, c_index_score = knn_loocv_with_cindex(X, y, k=10)
    print(f"C-index score: {c_index_score:.4f}")

    # 3. Visualization
    plot_true_vs_predicted(y, predictions)
    plot_residuals(y, predictions)
```

```
    # Additional analysis: Distribution of predictions
    plt.figure(figsize=(10, 5))
    plt.subplot(1, 2, 1)
    plt.hist(y, bins=20, alpha=0.7, label='True')
    plt.hist(predictions, bins=20, alpha=0.7, label='Predicted')
    plt.xlabel('Affinity')
    plt.ylabel('Frequency')
    plt.title('Distribution of True and Predicted Affinities')
    plt.legend()

    plt.subplot(1, 2, 2)
    plt.scatter(range(len(y)), y, label='True', alpha=0.5)
    plt.scatter(range(len(predictions)), predictions, label='Predicted',
alpha=0.5)
    plt.xlabel('Sample Index')
    plt.ylabel('Affinity')
    plt.title('True vs Predicted Affinities by Sample')
    plt.legend()
    plt.tight_layout()
    plt.show()
```

Explanation of the Code:

1. Data Loading and Preprocessing

- `load_and_preprocess_data()`:
  - Loads the three data files using pandas
  - Standardizes the features (mean=0, std=1) which is crucial for distance-based algorithms like KNN
  - Returns the features (X), targets (y), and pair identifiers

2. KNN with Leave-One-Out Cross-Validation

- `knn_loocv_with_cindex()`:
  - Initializes a KNN regressor with k=10 neighbors
  - Uses LeaveOneOut cross-validation which trains on all but one sample and tests on the left-out sample
  - Makes predictions for each left-out sample
  - Calculates the C-index score comparing true and predicted values

3. C-index Calculation

- `c_index()`:

- Custom implementation of the concordance index
- Measures the proportion of all pairs of samples where the predicted order matches the true order
- Returns a value between 0 (worst) and 1 (best)

4. Visualization

- `plot_true_vs_predicted()`:
  - Scatter plot comparing true vs predicted values with a perfect prediction line
- `plot_residuals()`:
  - Shows the errors (residuals) of predictions against predicted values
- Additional plots show:
  - Distribution comparison of true and predicted values
  - Sample-by-sample comparison of true and predicted values

Key Points:

1. **Standardization**: Essential for KNN as it's distance-based
2. **LOOCV**: Provides a robust evaluation by using each sample as a test case exactly once
3. **C-index**: Appropriate metric for evaluating ranking quality in regression problems
4. **Visualization**: Helps understand model performance and identify potential issues

To use this code, replace the file paths with your actual data file paths. The code assumes your data files are in CSV format without headers. If your files have different formats, you may need to adjust the loading code accordingly.

User
Previously you wrote a baseline KNN with 10 neighbours for the protein drug affinity task. But i think the validation is not correct. Currently it is using pair level leave one out and scales the full dataset before the cross validation.

what do you think which validation techniques we can use here to train such a model which can predict unseen data correctly?

Mistral
You're absolutely right to question the validation approach. The current implementation has two main issues:

1. **Data Leakage in Standardization**: Scaling the entire dataset before cross-validation means information from the test fold leaks into the training fold through the mean/std calculation.

2. **Pair-Level LOOCV**: While leave-one-out is thorough, it doesn't properly simulate the real-world scenario where we want to predict affinities for completely new protein-drug pairs.

Better Validation Approaches

For protein-drug affinity prediction, we need validation strategies that:

1. Prevent data leakage
2. Simulate real-world scenarios
3. Account for the paired nature of the data

Here are improved approaches:

1. Protein-Structured Cross-Validation

```
from sklearn.model_selection import GroupKFold

def protein_group_cv(X, y, pairs_df, n_splits=5):
    """
    Cross-validation where all pairs involving the same protein are in the
same fold.
    This simulates predicting affinities for new drugs against known proteins.
    """
    # Create groups based on protein_id
    groups = pairs_df['protein_id'].values

    # Initialize GroupKFold
    gkf = GroupKFold(n_splits=n_splits)

    # Store predictions
    predictions = np.zeros_like(y)

    for train_idx, test_idx in gkf.split(X, y, groups):
        X_train, X_test = X[train_idx], X[test_idx]
        y_train = y[train_idx]

        # Standardize within the training fold only
        scaler = StandardScaler()
        X_train_scaled = scaler.fit_transform(X_train)
        X_test_scaled = scaler.transform(X_test)

        # Train and predict
        knn = KNeighborsRegressor(n_neighbors=10)
        knn.fit(X_train_scaled, y_train)
        predictions[test_idx] = knn.predict(X_test_scaled)
```

```
    return predictions
```

2. Drug-Structured Cross-Validation

```
def drug_group_cv(X, y, pairs_df, n_splits=5):
    """
    Cross-validation where all pairs involving the same drug are in the same
fold.
    This simulates predicting affinities for new proteins against known drugs.
    """
    # Create groups based on drug_id
    groups = pairs_df['drug_id'].values

    # Rest is identical to protein_group_cv
    ...
```

3. Cold-Start Validation (Most Realistic)

```
def cold_start_validation(X, y, pairs_df, n_splits=5):
    """
    Simulates the most challenging real-world scenario:
    - Train on all known protein-drug pairs
    - Test on completely new proteins or drugs
    """
    # Get unique proteins and drugs
    unique_proteins = pairs_df['protein_id'].unique()
    unique_drugs = pairs_df['drug_id'].unique()

    # For protein cold-start:
    protein_kf = KFold(n_splits=n_splits)
    protein_predictions = np.zeros_like(y)

    for train_prot_idx, test_prot_idx in protein_kf.split(unique_proteins):
        # Get all pairs not involving test proteins
        train_mask =
~pairs_df['protein_id'].isin(unique_proteins[test_prot_idx])
        test_mask =
pairs_df['protein_id'].isin(unique_proteins[test_prot_idx])

        X_train, X_test = X[train_mask], X[test_mask]
        y_train = y[train_mask]

        # Standardize and train
```

```
        scaler = StandardScaler()
        X_train_scaled = scaler.fit_transform(X_train)
        X_test_scaled = scaler.transform(X_test)

        knn = KNeighborsRegressor(n_neighbors=10)
        knn.fit(X_train_scaled, y_train)
        protein_predictions[test_mask] = knn.predict(X_test_scaled)

    # Similarly for drug cold-start
    ...
```

4. Time-Based Validation (If Available)

If you have temporal information about when measurements were made:

```
def time_based_validation(X, y, pairs_df, time_column):
    """
    Sort by time and use earlier measurements for training,
    later ones for testing.
    """
    # Sort by time
    pairs_df = pairs_df.sort_values(time_column)
    X = X[pairs_df.index]
    y = y[pairs_df.index]

    # Split at a certain time point
    split_idx = int(0.8 * len(X))
    X_train, X_test = X[:split_idx], X[split_idx:]
    y_train = y[:split_idx]

    # Standardize and train
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    knn = KNeighborsRegressor(n_neighbors=10)
    knn.fit(X_train_scaled, y_train)
    predictions = knn.predict(X_test_scaled)

    return predictions
```

Recommended Approach

For your specific case with 77 proteins and 59 drugs, I recommend:

1. **Primary Validation**: Protein-structured CV (most common in drug discovery)
2. **Secondary Validation**: Drug-structured CV
3. **Final Test**: Cold-start validation (if you have enough data)

Here's how to implement the recommended approach:

```
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

def robust_knn_validation(X, y, pairs_df):
    """
    Comprehensive validation with proper scaling and realistic splits
    """
    # 1. Protein-structured CV
    print("Running protein-structured CV...")
    protein_pred = protein_group_cv(X, y, pairs_df)
    protein_cindex = c_index(y, protein_pred)
    print(f"Protein-structured C-index: {protein_cindex:.4f}")

    # 2. Drug-structured CV
    print("\nRunning drug-structured CV...")
    drug_pred = drug_group_cv(X, y, pairs_df)
    drug_cindex = c_index(y, drug_pred)
    print(f"Drug-structured C-index: {drug_cindex:.4f}")

    # 3. Cold-start validation (protein)
    print("\nRunning protein cold-start validation...")
    cold_prot_pred = cold_start_validation(X, y, pairs_df)
    cold_prot_cindex = c_index(y, cold_prot_pred)
    print(f"Protein cold-start C-index: {cold_prot_cindex:.4f}")

    # Visualize results
    plt.figure(figsize=(12, 4))
    plt.subplot(1, 3, 1)
    plt.scatter(y, protein_pred, alpha=0.5)
    plt.title(f'Protein-structured\nC-index: {protein_cindex:.3f}')

    plt.subplot(1, 3, 2)
    plt.scatter(y, drug_pred, alpha=0.5)
    plt.title(f'Drug-structured\nC-index: {drug_cindex:.3f}')

    plt.subplot(1, 3, 3)
    plt.scatter(y, cold_prot_pred, alpha=0.5)
    plt.title(f'Cold-start\nC-index: {cold_prot_cindex:.3f}')
    plt.tight_layout()
```

```
    plt.show()

    return {
        'protein_structured': protein_cindex,
        'drug_structured': drug_cindex,
        'cold_start': cold_prot_cindex
    }
```

Key Improvements:

1. **No Data Leakage**: Standardization is performed within each training fold only
2. **Realistic Scenarios**: Different validation strategies test different generalization capabilities
3. **Comprehensive Evaluation**: Multiple validation approaches give a complete picture of model performance
4. **Proper Scaling**: Using pipelines ensures scaling is properly applied

This approach will give you much more reliable estimates of how your model will perform on truly unseen data in real-world scenarios.

User
okay lets go with the Leave-One-Drug-Out (LODO) and Leave-One-Protein-Out (LOPO) techniques for the validation and generate me code for these.

Mistral

Leave-One-Drug-Out (LODO) and Leave-One-Protein-Out (LOPO) Validation

Here's a complete implementation of LODO and LOPO cross-validation for your protein-drug affinity prediction task:

```
import numpy as np
import pandas as pd
from sklearn.neighbors import KNeighborsRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
import matplotlib.pyplot as plt
from collections import defaultdict

# Custom C-index function
def c_index(y_true, y_pred):
    y_true = np.asarray(y_true)
    y_pred = np.asarray(y_pred)
```

```
    concordant = 0
    permissible = 0

    for i in range(len(y_true)):
        for j in range(i+1, len(y_true)):
            if y_true[i] != y_true[j]:
                permissible += 1
                if (y_pred[i] > y_pred[j] and y_true[i] > y_true[j]) or \
                   (y_pred[i] < y_pred[j] and y_true[i] < y_true[j]):
                    concordant += 1

    return concordant / permissible if permissible > 0 else 0.0

# Load and preprocess data
def load_data(input_file, output_file, pairs_file):
    X = pd.read_csv(input_file, header=None).values
    y = pd.read_csv(output_file, header=None).values.ravel()
    pairs_df = pd.read_csv(pairs_file, header=None, names=['protein_id',
'drug_id'])

    return X, y, pairs_df

# Leave-One-Drug-Out Cross-Validation
def lodo_cv(X, y, pairs_df, k=10):
    """
    Leave-One-Drug-Out Cross-Validation
    Each fold leaves out all pairs involving one drug
    """
    # Get unique drugs and create mapping
    unique_drugs = pairs_df['drug_id'].unique()
    drug_to_indices = defaultdict(list)

    for idx, drug_id in enumerate(pairs_df['drug_id']):
        drug_to_indices[drug_id].append(idx)

    # Initialize predictions array
    predictions = np.zeros_like(y)

    for drug_id in unique_drugs:
        # Get indices for current drug (test set)
        test_indices = drug_to_indices[drug_id]

        # Get training set (all other drugs)
        train_indices = [i for i in range(len(y)) if i not in test_indices]

        X_train, X_test = X[train_indices], X[test_indices]
```

```python
        y_train = y[train_indices]

        # Create pipeline with scaling and KNN
        model = make_pipeline(
            StandardScaler(),
            KNeighborsRegressor(n_neighbors=k)
        )

        # Train and predict
        model.fit(X_train, y_train)
        predictions[test_indices] = model.predict(X_test)

    return predictions

# Leave-One-Protein-Out Cross-Validation
def lopo_cv(X, y, pairs_df, k=10):
    """
    Leave-One-Protein-Out Cross-Validation
    Each fold leaves out all pairs involving one protein
    """
    # Get unique proteins and create mapping
    unique_proteins = pairs_df['protein_id'].unique()
    protein_to_indices = defaultdict(list)

    for idx, protein_id in enumerate(pairs_df['protein_id']):
        protein_to_indices[protein_id].append(idx)

    # Initialize predictions array
    predictions = np.zeros_like(y)

    for protein_id in unique_proteins:
        # Get indices for current protein (test set)
        test_indices = protein_to_indices[protein_id]

        # Get training set (all other proteins)
        train_indices = [i for i in range(len(y)) if i not in test_indices]

        X_train, X_test = X[train_indices], X[test_indices]
        y_train = y[train_indices]

        # Create pipeline with scaling and KNN
        model = make_pipeline(
            StandardScaler(),
            KNeighborsRegressor(n_neighbors=k)
        )
```

```
        # Train and predict
        model.fit(X_train, y_train)
        predictions[test_indices] = model.predict(X_test)

    return predictions

# Visualization functions
def plot_results(y_true, y_pred_lodo, y_pred_lopo):
    plt.figure(figsize=(15, 5))

    # LODO results
    plt.subplot(1, 3, 1)
    plt.scatter(y_true, y_pred_lodo, alpha=0.5)
    plt.plot([y_true.min(), y_true.max()], [y_true.min(), y_true.max()], 'k--
', lw=2)
    plt.xlabel('True Affinity')
    plt.ylabel('Predicted Affinity')
    plt.title(f'LODO\nC-index: {c_index(y_true, y_pred_lodo):.4f}')

    # LOPO results
    plt.subplot(1, 3, 2)
    plt.scatter(y_true, y_pred_lopo, alpha=0.5)
    plt.plot([y_true.min(), y_true.max()], [y_true.min(), y_true.max()], 'k--
', lw=2)
    plt.xlabel('True Affinity')
    plt.ylabel('Predicted Affinity')
    plt.title(f'LOPO\nC-index: {c_index(y_true, y_pred_lopo):.4f}')

    # Combined residuals
    plt.subplot(1, 3, 3)
    residuals_lodo = y_true - y_pred_lodo
    residuals_lopo = y_true - y_pred_lopo
    plt.scatter(y_pred_lodo, residuals_lodo, alpha=0.5, label='LODO')
    plt.scatter(y_pred_lopo, residuals_lopo, alpha=0.5, label='LOPO')
    plt.axhline(y=0, color='r', linestyle='-')
    plt.xlabel('Predicted Affinity')
    plt.ylabel('Residuals')
    plt.title('Residual Analysis')
    plt.legend()

    plt.tight_layout()
    plt.show()

# Main execution
if __name__ == "__main__":
    # Load data
```

```
    input_file = 'input.data'
    output_file = 'output.data'
    pairs_file = 'pairs.data'

    X, y, pairs_df = load_data(input_file, output_file, pairs_file)
    print(f"Data loaded. Features shape: {X.shape}, Targets shape: {y.shape}")
    print(f"Unique proteins: {pairs_df['protein_id'].nunique()}")
    print(f"Unique drugs: {pairs_df['drug_id'].nunique()}")

    # Run LODO CV
    print("\nRunning Leave-One-Drug-Out Cross-Validation...")
    lodo_predictions = lodo_cv(X, y, pairs_df, k=10)
    lodo_cindex = c_index(y, lodo_predictions)
    print(f"LODO C-index: {lodo_cindex:.4f}")

    # Run LOPO CV
    print("\nRunning Leave-One-Protein-Out Cross-Validation...")
    lopo_predictions = lopo_cv(X, y, pairs_df, k=10)
    lopo_cindex = c_index(y, lopo_predictions)
    print(f"LOPO C-index: {lopo_cindex:.4f}")

    # Visualize results
    plot_results(y, lodo_predictions, lopo_predictions)

    # Additional analysis: Performance by protein/drug
    # For LODO - performance by drug
    drug_performance = defaultdict(list)
    for idx, drug_id in enumerate(pairs_df['drug_id']):
        drug_performance[drug_id].append((y[idx], lodo_predictions[idx]))

    print("\nLODO Performance by Drug (top 5 best and worst):")
    drug_scores = {drug: c_index([true for true, _ in pairs],
                                 [pred for _, pred in pairs])
                   for drug, pairs in drug_performance.items()}
    sorted_drugs = sorted(drug_scores.items(), key=lambda x: x[1],
reverse=True)
    for drug, score in sorted_drugs[:5] + sorted_drugs[-5:]:
        print(f"Drug {drug}: C-index = {score:.4f}")

    # For LOPO - performance by protein
    protein_performance = defaultdict(list)
    for idx, protein_id in enumerate(pairs_df['protein_id']):
        protein_performance[protein_id].append((y[idx],
lopo_predictions[idx]))

    print("\nLOPO Performance by Protein (top 5 best and worst):")
```

```
    protein_scores = {prot: c_index([true for true, _ in pairs],
                                    [pred for _, pred in pairs])
                      for prot, pairs in protein_performance.items()}
    sorted_proteins = sorted(protein_scores.items(), key=lambda x: x[1],
reverse=True)
    for prot, score in sorted_proteins[:5] + sorted_proteins[-5:]:
        print(f"Protein {prot}: C-index = {score:.4f}")
```

Explanation of the Implementation:

1. Data Loading

- The `load_data()` function reads your three input files
- Returns features (X), targets (y), and pair identifiers

2. Leave-One-Drug-Out (LODO) CV

- `lodo_cv()`:
  - Creates a mapping of each drug to all its associated pairs
  - For each drug, leaves out all its pairs as the test set
  - Trains on all other pairs (involving other drugs)
  - Uses a pipeline with StandardScaler and KNN to prevent data leakage
  - Returns predictions for all pairs

3. Leave-One-Protein-Out (LOPO) CV

- `lopo_cv()`:
  - Similar to LODO but operates on proteins instead of drugs
  - For each protein, leaves out all its pairs as the test set
  - Trains on all other pairs (involving other proteins)

4. Visualization

- `plot_results()`:
  - Shows true vs predicted plots for both LODO and LOPO
  - Includes a residual plot comparing both methods
  - Displays C-index scores in the titles

5. Performance Analysis

- After running both CV methods, the code analyzes:
  - Performance by individual drug (for LODO)

- Performance by individual protein (for LOPO)
- Shows top 5 and bottom 5 performers for each

Key Advantages of This Approach:

1. **Realistic Evaluation**:
   - LODO simulates predicting affinities for a completely new drug
   - LOPO simulates predicting affinities for a completely new protein
2. **No Data Leakage**:
   - StandardScaler is included in the pipeline and fitted only on training data
   - Each fold is completely independent
3. **Comprehensive Analysis**:
   - Overall performance metrics (C-index)
   - Per-drug and per-protein performance breakdown
   - Visual comparison of both methods
4. **Practical Insights**:
   - Identifies which drugs/proteins are easier/harder to predict
   - Helps understand model strengths and weaknesses

Interpretation of Results:

- **LODO C-index**: Shows how well the model generalizes to new drugs
- **LOPO C-index**: Shows how well the model generalizes to new proteins
- **Difference between scores**: Indicates whether the model is better at generalizing to new drugs or new proteins
- **Per-drug/protein scores**: Reveals which specific molecules are most/least predictable

This implementation provides a rigorous evaluation of your model's ability to handle real-world scenarios where you need to predict affinities for new drugs or proteins.

User
don't you think type D cross-validation (unseen drug - unseen target) should be implemented as per the requirement? Please give me reason how it can be beneficial in our case

Mistral
You're absolutely right to consider **Type D cross-validation** (unseen drug - unseen target) for your protein-drug affinity prediction task. This is the most rigorous and realistic validation approach for drug discovery applications. Let me explain why it's beneficial and how to implement it.

Why Type D Cross-Validation is Beneficial

1. Real-World Scenario Simulation

- In actual drug discovery, you often need to predict affinities for:
  - Completely new drugs against new protein targets
  - This is exactly what Type D CV tests

2. Most Challenging Test

- Type A (random split): Too optimistic, doesn't test generalization
- Type B (LODO): Tests new drugs against known proteins
- Type C (LOPO): Tests new proteins against known drugs
- **Type D**: Tests both new drugs AND new proteins simultaneously

3. Identifies True Model Limitations

- Reveals if your model can:
  - Learn generalizable patterns about molecular interactions
  - Handle completely novel chemical spaces
  - Generalize to new biological targets

4. Practical Benefits for Your Case

With 77 proteins and 59 drugs:

- You have enough data to create meaningful Type D splits
- Helps identify if your model is memorizing patterns or truly learning
- Guides feature engineering to improve generalization

Implementation of Type D Cross-Validation

Here's how to implement it for your dataset:

```
from sklearn.model_selection import train_test_split
from itertools import product
import random

def type_d_cv(X, y, pairs_df, test_size=0.2, random_state=None):
    """
    Type D Cross-Validation: Unseen drugs against unseen proteins
    Creates a test set with completely new drug-protein pairs
    """
    if random_state is not None:
        random.seed(random_state)
```

```
        np.random.seed(random_state)

    # Get unique proteins and drugs
    unique_proteins = pairs_df['protein_id'].unique()
    unique_drugs = pairs_df['drug_id'].unique()

    # Split proteins and drugs into train/test sets
    train_proteins, test_proteins = train_test_split(
        unique_proteins, test_size=test_size, random_state=random_state
    )
    train_drugs, test_drugs = train_test_split(
        unique_drugs, test_size=test_size, random_state=random_state
    )

    # Create train and test masks
    train_mask = (
        pairs_df['protein_id'].isin(train_proteins) &
        pairs_df['drug_id'].isin(train_drugs)
    )
    test_mask = (
        pairs_df['protein_id'].isin(test_proteins) &
        pairs_df['drug_id'].isin(test_drugs)
    )

    # Get the actual data
    X_train, X_test = X[train_mask], X[test_mask]
    y_train = y[train_mask]

    # Create and train model
    model = make_pipeline(
        StandardScaler(),
        KNeighborsRegressor(n_neighbors=10)
    )
    model.fit(X_train, y_train)

    # Predict on test set
    y_pred = model.predict(X_test)

    return y_test, y_pred, test_mask

def repeated_type_d_cv(X, y, pairs_df, n_repeats=5, test_size=0.2):
    """
    Perform Type D CV multiple times for more reliable results
    """
    all_c_indices = []
```

```
    for i in range(n_repeats):
        print(f"\nType D CV iteration {i+1}/{n_repeats}")
        y_test, y_pred, test_mask = type_d_cv(
            X, y, pairs_df, test_size=test_size, random_state=i
        )
        c_idx = c_index(y_test, y_pred)
        all_c_indices.append(c_idx)
        print(f"Iteration {i+1} C-index: {c_idx:.4f}")

    print(f"\nAverage C-index: {np.mean(all_c_indices):.4f} ±
{np.std(all_c_indices):.4f}")

    return all_c_indices

def analyze_type_d_results(X, y, pairs_df, test_mask):
    """
    Analyze the Type D test set characteristics
    """
    test_pairs = pairs_df[test_mask]
    print("\nType D Test Set Analysis:")
    print(f"Number of test pairs: {len(test_pairs)}")
    print(f"Unique proteins in test: {test_pairs['protein_id'].nunique()}")
    print(f"Unique drugs in test: {test_pairs['drug_id'].nunique()}")

    # Compare feature distributions
    train_mask = ~test_mask
    X_train, X_test = X[train_mask], X[test_mask]

    plt.figure(figsize=(12, 5))
    for i in range(min(5, X.shape[1])):  # Plot first 5 features
        plt.subplot(2, 3, i+1)
        plt.hist(X_train[:,i], bins=20, alpha=0.5, label='Train')
        plt.hist(X_test[:,i], bins=20, alpha=0.5, label='Test')
        plt.title(f'Feature {i} Distribution')
        plt.legend()
    plt.tight_layout()
    plt.show()
```

How to Use This in Your Workflow

```
# Main execution with all validation types
if __name__ == "__main__":
    # Load data
    X, y, pairs_df = load_data('input.data', 'output.data', 'pairs.data')
```

```python
# Run all validation types
print("="*50)
print("COMPREHENSIVE MODEL VALIDATION")
print("="*50)

# 1. Type B (LODO)
print("\n1. Type B - Leave-One-Drug-Out (LODO)")
lodo_pred = lodo_cv(X, y, pairs_df)
print(f"LODO C-index: {c_index(y, lodo_pred):.4f}")

# 2. Type C (LOPO)
print("\n2. Type C - Leave-One-Protein-Out (LOPO)")
lopo_pred = lopo_cv(X, y, pairs_df)
print(f"LOPO C-index: {c_index(y, lopo_pred):.4f}")

# 3. Type D (Unseen drug - unseen protein)
print("\n3. Type D - Unseen Drug vs Unseen Protein")
type_d_scores = repeated_type_d_cv(X, y, pairs_df, n_repeats=5)

# 4. Compare all methods
plt.figure(figsize=(10, 6))
methods = ['LODO', 'LOPO', 'Type D (avg)']
scores = [
    c_index(y, lodo_pred),
    c_index(y, lopo_pred),
    np.mean(type_d_scores)
]
errors = [0, 0, np.std(type_d_scores)]

plt.bar(methods, scores, yerr=errors,
        capsize=10, color=['blue', 'orange', 'green'])
plt.ylabel('C-index')
plt.title('Model Performance Across Validation Types')
plt.ylim(0, 1)
plt.grid(True, axis='y', linestyle='--', alpha=0.7)
plt.show()

# 5. Analyze Type D test set
_, _, test_mask = type_d_cv(X, y, pairs_df, test_size=0.2, random_state=0)
analyze_type_d_results(X, y, pairs_df, test_mask)
```

Key Insights from Type D Validation

1. **Performance Drop Expectation**:
   - Type D will typically show lower performance than LODO/LOPO

- This is expected and reveals the true generalization capability

2. **Feature Analysis**:
   - The distribution comparison shows if your test set is representative
   - Large differences suggest potential issues with feature engineering
3. **Model Improvement Directions**:
   - If Type D performance is much worse:
     - Your model may be overfitting to specific protein/drug families
     - Need more diverse training data or better features
   - If all types perform similarly:
     - Your model has learned generalizable patterns
4. **Practical Implications**:
   - High Type D score means your model can:
     - Predict affinities for completely new drug candidates
     - Suggest new targets for existing drugs
     - Guide experimental validation efforts

Recommendations for Your Project

1. **Start with Type B/C**:
   - First run LODO and LOPO to establish baselines
   - These are less strict but still valuable
2. **Then Implement Type D**:
   - Use the repeated version for statistical significance
   - Analyze the test set characteristics
3. **Compare All Results**:
   - The performance drop from Type B/C to Type D shows:
     - How much your model relies on seen proteins/drugs
     - The true generalization capability
4. **Iterative Improvement**:
   - Use Type D results to guide feature engineering
   - Try to reduce the gap between Type C and Type D performance

This comprehensive validation approach will give you the most realistic assessment of your model's capabilities for actual drug discovery applications.